```
array(2) {
  [0]=>
  object(Monk)#1 (0) {
  }
  [1]=>
  object(Barbarian)#2 (0) {
  }
}
```

Hopefully you'll have also noticed how I chose to access the attributes: once on the way in, rather than every time `getHealers()` or `getFighters()` is called. Using attributes and reflection naturally comes with a small performance hit, so you might need to make decisions when using them in performance-sensitive areas of your code (keep them out of loops or heavily used functions).

Attributes can be passed data as we saw when using `#[Deprecated]`, and because they are fully formed PHP classes you can also implement methods and whatever logic you like inside them. `ReflectionAttribute::newInstance()` will return a new instance of the attribute class; from there you can use it like any other object. But don't. Attributes are about metadata, not logic; they should describe the code they mark, not serve as code themselves. I hope this quick introduction to reflection and attributes has been useful; there's more to be said on the topic, but there's only so much space in this book. *PHP 8 Objects, Patterns, and Practice* covers these aspects of object-oriented PHP in more detail!

Summary

This chapter contains enough information so that any OOP code encountered in the following chapters will make sense. We've covered the essentials of object creation and use, how to define classes, the difference between objects and classes, and how to access static and non-static members.

We've also touched upon some of the core concepts of OOP, such as inheritance, abstraction, and composing classes from traits, which I will be referring back to in Chapter 10, "Programming with Objects." These mechanisms might not appear to be particularly game-changing when viewed in isolation, but in the context of object-oriented analysis and design, their essential nature will emerge. I promise!